

Mini-Project 1: SMoL-X Programming Language Implementation Project

- **Submission deadline:** February 26 at 11:59 PM
- **Presentation deadline:** February 26 at class time (every participant should be present to receive the grade, **no presentation, no grade**)
- **Note:** Every single team member must upload the project via Canvas

Project Information

Based on: Programming Languages: Application and Interpretation (PLAI), Version 3.2.5

Material: Pages 1-57 (Chapters: Evaluation, Parsing, Conditionals, Local Binding, Substitution, and Caching Substitution)

Duration: 2 weeks

Group Size: 3-4 students per group

Project Overview

In this project, each group will design and implement a small programming language called **SMoL-X** (Small Language with eXtensions). The goal is to explore two different implementation strategies for evaluation: a **substitution-based interpreter** and an **environment-based interpreter**. You will compare these two approaches both conceptually and experimentally, gaining deep insight into how programming languages are implemented in practice.

This project simulates the work of a language design team evaluating implementation strategies. Your final submission should demonstrate both technical correctness and conceptual understanding of fundamental programming language concepts.

Learning Objectives

By completing this project, students will be able to:

1. **Understand substitution-based evaluation** and explain its theoretical foundation
2. **Implement abstract syntax trees (ASTs)** and parsers for a small language
3. **Explain static scoping** formally and implement it correctly
4. **Analyze inefficiencies** in naive substitution through empirical measurement
5. **Justify the use of environments** in real language implementations
6. **Design and execute experiments** to compare implementation strategies
7. **Write technical documentation** explaining language design decisions

Required Readings

Students must read and understand the following sections from PLAI Version 3.2.5:

Week 1 - Substitution Interpreter

- **Pages 16-30:** Evaluation on Paper, Representing Arithmetic, Evaluating Arithmetic
 - Understanding evaluators and terminology
 - Abstract syntax trees
 - Type definitions in Plait
- **Pages 31-36:** Parsing: From Source to ASTs
 - S-expressions
 - Converting concrete syntax to AST
- **Pages 37-45:** Evaluating Conditionals
 - Extending the AST
 - Truthy/falsy values
 - The Value datatype
- **Pages 46-53:** Evaluating Local Binding (Part 1)
 - Syntax for local binding
 - The meaning of `(let1)`
 - **Static scoping**
 - Substitution-based evaluation

Week 2 - Environment Interpreter

- **Pages 54-56:** Caching Substitution
 - Environment-based evaluation
 - Hash tables for environments
 - Lookup and extend operations
 - Comparison with substitution

Additional Recommended Reading

- Review the entire evaluation section (pages 16-56) to understand the progression from substitution to environments

- Pay special attention to the exercises and "Do Now!" sections, as they contain important conceptual questions

Tools and Setup

Required Software

1. DrRacket with Plait

- Download from: <https://racket-lang.org/>
- Version: 8.0 or later recommended
- Install the Plait language: Run `raco pkg install plait` in terminal/command prompt
- All code must use `#lang plait` at the top of the file

2. Text Editor (optional, for report writing)

- Any plain text editor or LaTeX editor
- Microsoft Word, Google Docs, or similar for the final report

Setting Up DrRacket

1. Install DrRacket from the official website
2. Open DrRacket
3. Install Plait by running in the terminal:

```
bash  
  
raco pkg install plait
```

4. Create a new file and add `#lang plait` as the first line
5. Test your setup with a simple program:

```
racket  
  
#lang plait  
  
(define (hello name)  
  (string-append "Hello, " name "!"))  
  
(test (hello "World") "Hello, World!")
```

Plait Quick Reference

Key Plait constructs you'll use:

```
racket

;; Type definitions
(define-type Exp
[num (n : Number)]
[plus (left : Exp) (right : Exp)])

;; Pattern matching
(type-case Exp e
[(num n) ...]
[(plus l r) ...])

;; Hash tables (for environments)
(define mt-env (hash empty))
(hash-set env 'x 42)
(hash-ref env 'x)

;; Testing
(test (calc (num 5)) 5)
(test (calc (plus (num 2) (num 3))) 5)
```

Language Specification

Your language **SMoL-X** must support the following constructs:

Required Language Features

1. **Numbers** - Integer literals (e.g., `1`, `42`, `-5`)
2. **Booleans** - `true` and `false`
3. **Addition** - Binary addition operator
4. **Conditionals** - `if` expressions with test, then-branch, and else-branch
5. **Single-variable local binding** - `let1` for introducing one new variable
6. **Variables** - Variable references

`let1`

7. **Nested bindings** - expressions can appear inside other expressions

8. **Shadowing** - Inner bindings can shadow outer bindings with the same name

Grammar Definition

You must define a clear grammar for your language in **Backus-Naur Form (BNF)** in your report. Here is an example structure (you should refine this):

```
bnf

<expr> ::= <number>
        | <boolean>
        | <variable>
        | (+ <expr> <expr>)
        | (if <expr> <expr> <expr>)
        | (let1 (<variable> <expr>) <expr>)

<number> ::= integer literals
<boolean> ::= true | false
<variable> ::= identifier (symbol)
```

Example Programs

Your language should be able to evaluate programs like:

```
racket

;; Simple arithmetic
(+ 2 3) ;=> 5

;; Conditionals
(if true 10 20) ;=> 10

;; Local binding
(let1 (x 5)
  (+ x 3)) ;=> 8

;; Nested bindings
(let1 (x 10)
  (let1 (y 20)
    (+ x y))) ;=> 30

;; Shadowing
(let1 (x 5)
  (let1 (x 10)
    x)) ;=> 10
```

Phase 1: Substitution Interpreter (Week 1)

Deliverables for Week 1

By the end of the first week, your group must implement:

1. Abstract syntax tree (AST) using Racket structs (define-type)
2. Parser that converts s-expressions into your AST representation
3. Substitution-based evaluator
4. Instrumentation to count substitution operations
5. Initial test suite (at least 15 tests)

1.1 Abstract Syntax Tree

Define your AST using `define-type`. Your AST should include constructors for:

- Numbers (`numE`)
- Booleans (`boolE`)
- Variables (`varE`)
- Addition (`plusE`)
- Conditionals (`ifE`)
- Local binding (`let1E`)

Example structure:

```
racket

(define-type Exp
  [numE (n : Number)]
  [boolE (b : Boolean)]
  [varE (name : Symbol)]
  [plusE (left : Exp) (right : Exp)]
  [ifE (test : Exp) (then : Exp) (else : Exp)]
  [let1E (var : Symbol) (val : Exp) (body : Exp)])
```

1.2 Parser Implementation

Your parser must convert s-expressions into AST nodes. Use the following helper functions:

- `s-exp-number?` - Check if s-expression is a number

- ♦ `s-exp-symbol?` - Check if s-expression is a symbol

- `s-exp-list?` - Check if s-expression is a list
- `s-exp->number` - Extract number from s-expression
- `s-exp->symbol` - Extract symbol from s-expression
- `s-exp->list` - Extract list from s-expression

Example parser structure:

```
racket

(define (parse s)
  (cond
    [(s-exp-number? s)
     (numE (s-exp->number s))]
    [(s-exp-symbol? s)
     (varE (s-exp->symbol s))]
    [(s-exp-list? s)
     (let ([l (s-exp->list s)])
       (cond
         [(symbol=? '+ (s-exp->symbol (first l)))
          (plusE (parse (second l))
                  (parse (third l)))]
         ;; Add more cases for if, let1, etc.
         [else (error 'parse "unknown expression")]])))]))
```

1.3 Substitution Function

The substitution function is the **heart of your interpreter**. It must:

- Replace all free occurrences of a variable with a value
- Preserve static scoping
- **NOT** substitute into shadowed variables
- Handle all expression types correctly

Critical requirement: Your substitution must correctly handle shadowing:

```
racket

;; Example: substituting x with 5 in the expression
;; (let1 (x 10) (+ x 3))
;; Should produce: (let1 (x 10) (+ x 3))
;; NOT: (let1 (5 10) (+ 5 3))
```

Function signature:

```
racket
```

```
;; subst : Symbol Value Exp -> Exp
;; Replace all free occurrences of var with val in expr
(define (subst var val expr)
  (type-case Exp expr
    [(numE n) expr] ; no substitution in numbers
    [(varE x) (if (symbol=? x var) val expr)]
    [(plusE l r) (plusE (subst var val l)
                        (subst var val r))]
    ;; Handle other cases, being careful with let1E shadowing
  ))
```

1.4 Evaluator Implementation

Implement an evaluator using substitution:

```
racket
```

```
(define (eval-sub e)
  (type-case Exp e
    [(numE n) n]
    [(boolE b) b]
    [(varE s) (error 'eval "free variable")]
    [(plusE l r) (+ (eval-sub l) (eval-sub r))]
    [(ifE test then else)
     (if (eval-sub test)
         (eval-sub then)
         (eval-sub else))]
    [(let1E var val body)
     (eval-sub (subst var (eval-sub val) body))]))
```

1.5 Instrumentation

Add a counter to track substitution operations:

```
racket
```

```
;; Global counter
(define sub-count (box 0))

;; Reset counter
(define (reset-counter!)
  (set-box! sub-count 0))

;; Increment in subst function
(define (subst var val expr)
  (set-box! sub-count (+ 1 (unbox sub-count)))
  ;; ... rest of substitution logic
  )

;; Get current count
(define (get-sub-count)
  (unbox sub-count))
```

1.6 Testing Requirements (Week 1)

Include at least **15 tests** covering:

- Simple arithmetic (3 tests)
- Conditionals with true/false (3 tests)
- Variable binding and lookup (3 tests)
- Nested bindings (3 tests)
- Shadowing behavior (3 tests)

Example tests:

```
racket

(test (eval-sub (parse '5)) 5)
(test (eval-sub (parse '(+ 2 3))) 5)
(test (eval-sub (parse '(if true 10 20))) 10)
(test (eval-sub (parse '(let1 (x 5) x))) 5)
(test (eval-sub (parse '(let1 (x 5) (let1 (x 10) x)))) 10)
```

Phase 2: Environment Interpreter (Week 2)

Deliverables for Week 2

By the end of the second week, your group must implement:

1. Environment-based evaluator using hash tables
2. Lookup and extend operations for environments
3. Instrumentation to count environment lookups
4. Complete test suite (total of 25+ tests)
5. Three stress test programs for experimental analysis
6. Written report with analysis

2.1 Environment Data Structure

Define the environment type and basic operations:

```
racket

;; Type alias for environments
(define-type-alias Env (Hashof Symbol Value))

;; Empty environment
(define mt-env (hash empty))

;; Lookup a variable in the environment
(define (lookup s env)
  (type-case (Optionof Value) (hash-ref env s)
    [(none) (error 'lookup "variable not bound")]
    [(some v) v]))

;; Extend environment with a new binding
(define (extend env var val)
  (hash-set env var val))
```

2.2 Environment-Based Evaluator

Implement the evaluator that uses environments instead of substitution:

```
racket
```

```
(define (eval-env e env)
  (type-case Exp e
    [(numE n) n]
    [(boolE b) b]
    [(varE s) (lookup s env)]
    [(plusE l r) (+ (eval-env l env) (eval-env r env))]
    [(ifE test then else)
     (if (eval-env test env)
         (eval-env then env)
         (eval-env else env))]
    [(let1E var val body)
     (let ([new-env (extend env var (eval-env val env))])
       (eval-env body new-env))]))
```

Key differences from substitution:

1. The right-hand side of `(let1)` is evaluated **once**
2. The result is stored in the environment
3. Variable lookup uses the environment, not substitution
4. No text rewriting occurs

2.3 Instrumentation for Environments

Add a lookup counter:

```
racket
```

```

;; Global counter for lookups
(define lookup-count (box 0))

;; Reset counter
(define (reset-lookup-counter!)
  (set-box! lookup-count 0))

;; Modified lookup with counting
(define (lookup s env)
  (set-box! lookup-count (+ 1 (unbox lookup-count)))
  (type-case (Optionof Value) (hash-ref env s)
    [(none) (error 'lookup "variable not bound")]
    [(some v) v]))

;; Get current count
(define (get-lookup-count)
  (unbox lookup-count))

```

2.4 Preserving Static Scoping

Critical verification: Your environment-based evaluator must produce the same results as your substitution evaluator for all valid programs. This is a fundamental requirement.

Test static scoping explicitly:

```

racket

;; Static scoping test
(define static-test
  '(let1 (x 5)
    (let1 (f (lambda (y) (+ x y)))
      (let1 (x 10)
        (f 3)))))
;; Should evaluate to 8, not 13

```

Experimental Component

Required Stress Tests

Design and implement **three stress test programs** that demonstrate different performance characteristics:

1. Duplication Stress Test

Purpose: Demonstrate repeated evaluation under substitution

Example concept:

```
racket

;; This program causes the substitution interpreter to
;; evaluate the same expression multiple times
'(let1 (x (+ 1000 2000))
  (+ x x))

;; In substitution: (+ 1000 2000) is evaluated TWICE
;; In environment: (+ 1000 2000) is evaluated ONCE
```

Design a program that amplifies this effect dramatically. Consider deeply nested let-bindings where variables are used multiple times.

2. Shadowing Verification Test

Purpose: Confirm static scope is preserved in both interpreters

Example concept:

```
racket

;; This program tests that inner bindings correctly shadow outer ones
'(let1 (x 1)
  (let1 (y (+ x 10)) ; y = 11
    (let1 (x 5)      ; shadows outer x
      (let1 (z (+ x y)) ; z = 5 + 11 = 16 (not 5 + 15)
        z))))
```

Create a program with multiple levels of shadowing to verify correctness.

3. Deep Nesting Stress Test

Purpose: Stress the interpreter with deeply nested let-bindings

Example concept:

```
racket
```

;; Deeply nested bindings

```
'(let1 (a 1)
  (let1 (b (+ a 1))
    (let1 (c (+ b 1))
      (let1 (d (+ c 1))
        (let1 (e (+ d 1))
          e))))))
```

Create a program with 10-20 levels of nesting to measure performance differences.

Measurement Requirements

For each stress test, measure and record:

1. **Substitution count** (for substitution interpreter)
2. **Lookup count** (for environment interpreter)
3. **Execution time** (optional, but recommended)
4. **Result verification** (both must produce the same answer)

Expected Analysis

Your report should explain:

- **Why** the substitution interpreter performs more operations
- **Where** the duplication occurs in substitution
- **How** environments eliminate redundant work
- **When** environments provide the most benefit

Written Report

Report Structure (3-5 pages)

Your report must be well-organized, clearly written, and technically accurate. Use the following structure:

1. Language Definition and Grammar (0.5-1 page)

- Present your SMoL-X grammar in BNF notation
- Explain each construct briefly
- Provide 2-3 example programs with expected output
- Describe any design choices you made

2. Substitution Interpreter (1-1.5 pages)

- Explain the substitution algorithm
- Describe how you handle shadowing
- Show key code snippets (AST definition, substitution function)
- Discuss challenges encountered and how you solved them

3. Environment Interpreter (1-1.5 pages)

- Explain the environment-based approach
- Describe your environment data structure
- Show key code snippets (lookup, extend, eval-env)
- Explain how it differs from substitution

4. Experimental Setup and Results (1-1.5 pages)

For each stress test, include:

- Description of the test program
- Substitution count
- Lookup count
- Analysis of the results

Create a comparison table:

Test	Substitution Ops	Lookup Ops	Speedup Factor
Duplication	1000	50	20x
Shadowing	500	40	12.5x
Deep Nesting	2000	100	20x

5. Analysis and Conclusions (0.5-1 page)

Must address:

1. Why substitution can be inefficient:

- Explain the linear time complexity per binding
- Describe how repeated variable use causes duplication

- Give specific examples from your stress tests
2. **How caching substitution (environments) improves performance:**
 - Explain the space-time tradeoff
 - Describe constant-time lookup vs. linear-time substitution
 - Quantify the improvement from your experiments
 3. **Why environment-based evaluation is closer to real implementations:**
 - Discuss practical considerations (memory, speed)
 - Explain why modern interpreters use environments
 - Connect to what you learned about language implementation
 4. **Limitations of your implementation:**
 - What features would a real language need?
 - What optimizations could be added?
 - What did you learn from this project?

Deliverables

What to Submit

1. **Single Racket file** (`smol-x.rkt`) containing:
 - `#lang plait` declaration
 - AST definition
 - Parser implementation
 - Substitution-based interpreter with instrumentation
 - Environment-based interpreter with instrumentation
 - At least **25 tests** (use `test` form)
 - All helper functions
 - Clear comments explaining each section
2. **PDF report** (`report.pdf`) with:
 - Title page with group members' names
 - 3-5 pages of content (excluding title page)
 - All required sections (see Report Structure)
 - Tables and/or graphs showing experimental results

- References to PLAI textbook where appropriate

File Organization

```
submission/
├── smol-x.rkt    # Main implementation file
└── report.pdf    # Written analysis
```

Code Organization Template

```
racket
```

#lang plait

```
;; =====  
;; SMOl-X Language Implementation  
;; Group Members: [Names]  
;; =====
```

```
;; =====  
;; PART 1: ABSTRACT SYNTAX TREE  
;; =====
```

```
(define-type Exp  
  ;; Define your AST here  
)
```

```
;; =====  
;; PART 2: PARSER  
;; =====
```

```
(define (parse s)  
  ;; Parser implementation  
)
```

```
;; =====  
;; PART 3: SUBSTITUTION INTERPRETER  
;; =====
```

```
;; Substitution counter  
(define sub-count (box 0))
```

```
(define (reset-sub-counter!)  
  (set-box! sub-count 0))
```

```
(define (get-sub-count)  
  (unbox sub-count))
```

```
(define (subst var val expr)  
  ;; Substitution implementation  
)
```

```
(define (eval-sub e)  
  ;; Evaluator using substitution  
)
```

```
;; =====  
;; PART 4: ENVIRONMENT INTERPRETER  
;; =====
```

```
(define-type-alias Env (Hashof Symbol Value))
```

```
(define mt-env (hash empty))
```

```
;; Lookup counter
```

```
(define lookup-count (box 0))
```

```
(define (reset-lookup-counter!)
```

```
  (set-box! lookup-count 0))
```

```
(define (get-lookup-count)
```

```
  (unbox lookup-count))
```

```
(define (lookup s env)
```

```
  ;; Lookup implementation
```

```
)
```

```
(define (extend env var val)
```

```
  ;; Extend implementation
```

```
)
```

```
(define (eval-env e env)
```

```
  ;; Evaluator using environments
```

```
)
```

```
;; =====  
;; PART 5: STRESS TESTS  
;; =====
```

```
;; Duplication stress test
```

```
(define dup-test
```

```
  '(...))
```

```
;; Shadowing verification test
```

```
(define shadow-test
```

```
  '(...))
```

```
;; Deep nesting stress test
```

```
(define nest-test
```

```
'(...))
```

```
:: =====  
;; PART 6: TEST SUITE (25+ tests)  
;; =====
```

```
:: Parser tests  
(test (parse '5) (numE 5))  
;; ... more tests
```

```
:: Arithmetic tests  
(test (eval-sub (parse '(+ 2 3))) 5)  
;; ... more tests
```

```
:: Conditional tests  
;; ... tests
```

```
:: Binding tests  
;; ... tests
```

```
:: Shadowing tests  
;; ... tests
```

```
:: Consistency tests (sub vs env)  
(test (eval-sub (parse dup-test))  
      (eval-env (parse dup-test) mt-env))  
;; ... more tests
```

Evaluation Criteria

Your project will be evaluated on the following criteria:

Code Quality (40 points)

- **Correct substitution interpreter** (15 points)
 - Handles all language features
 - Correctly implements static scoping
 - Proper handling of shadowing
- **Correct environment interpreter** (15 points)
 - Produces same results as substitution

- Efficient lookup and extend
- Maintains static scoping
- **Code organization and clarity** (10 points)
 - Well-commented
 - Clear naming conventions
 - Logical structure

Testing (20 points)

- **Test coverage** (10 points)
 - At least 25 tests
 - Tests for all language features
 - Edge cases covered
- **Stress tests** (10 points)
 - Three distinct stress tests
 - Clear demonstration of performance differences
 - Correct instrumentation

Experimental Analysis (20 points)

- **Data collection** (8 points)
 - Accurate measurements
 - Complete results table
 - Clear presentation
- **Analysis depth** (12 points)
 - Explains why substitution is inefficient
 - Explains how environments improve performance
 - Connects to real-world implementations

Written Report (20 points)

- **Technical accuracy** (10 points)
 - Correct explanations of concepts
 - Accurate grammar definition
 - Sound reasoning in analysis

- **Writing quality** (10 points)
 - Clear, professional writing
 - Well-organized structure
 - Appropriate length (3-5 pages)

Timeline and Milestones

Week 1 Milestones

Day 1-2:

- Set up DrRacket and Plait
- Read PLAI pages 16-36
- Define AST structure
- Begin parser implementation

Day 3-4:

- Complete parser
- Read PLAI pages 37-53
- Implement substitution function
- Begin evaluator

Day 5-7:

- Complete substitution interpreter
- Add instrumentation
- Write initial 15 tests
- Debug and verify correctness

Week 2 Milestones

Day 1-2:

- Read PLAI pages 54-56
- Implement environment data structure
- Implement lookup and extend
- Begin environment evaluator

Day 3-4:

- Complete environment interpreter
- Add lookup instrumentation
- Verify both interpreters produce same results
- Design stress tests

Day 5-6:

- Implement and run stress tests
- Collect experimental data
- Write report sections 1-4

Day 7:

- Complete analysis section
- Final testing and debugging
- Proofread and submit

Common Pitfalls to Avoid

Technical Pitfalls

1. Incorrect shadowing in substitution:

- Don't substitute into variables bound by inner `(let1`
- Test this explicitly: `(let1 (x 5) (let1 (x 10) x))` should be `(10)`, not `(5)`

2. Evaluating the wrong expression in `let1`:

- Must evaluate `val` in the CURRENT environment
- Must evaluate body in the EXTENDED environment

3. Forgetting to reset counters:

- Reset counters before each test
- Don't accumulate counts across tests

4. Not testing equivalence:

- Both interpreters must produce the same results
- Test this for every program in your test suite

Report Pitfalls

1. Too much code, not enough explanation:

- Show key snippets, not entire functions
- Explain the algorithm, not just the code

2. Missing analysis:

- Don't just present numbers
- Explain WHY the numbers differ
- Connect to theoretical concepts from PLAI

3. Insufficient experimental data:

- Run all three stress tests
- Present results clearly in a table
- Discuss each test individually

Additional Resources

PLAI Online Resources

- Book website: <https://www.plai.org>
- Errata and updates available online
- Additional exercises and examples

Plait Documentation

- Plait language guide in DrRacket Help Desk
- Type system documentation
- Standard library reference

Academic Integrity

This is a **group project**. All work submitted must be your own group's work. You may:

- Discuss high-level concepts with other groups
- Ask the instructor for clarification
- Reference the PLAI textbook and course materials

You may **NOT**:

- Share code with other groups
- Copy code from online sources
- Submit work that is not your own

Any violations will be handled according to the university's academic integrity policies.

Frequently Asked Questions

Q: Can we add additional features to the language?

A: You may add optional extensions AFTER completing all required features, but they will not earn extra credit unless explicitly discussed with the instructor.

Q: What if our two interpreters produce different results?

A: This indicates a bug. Both interpreters must produce identical results for all valid programs. Debug by testing simple programs first, then gradually increase complexity.

Q: How detailed should the stress tests be?

A: They should be complex enough to show clear performance differences (at least 5-10x difference in operation counts). The duplication test is particularly important.

Q: Can we use features from Racket that aren't in Plait?

A: No. Your code must work in Plait (`(#lang plait)`). This ensures type safety and consistent behavior.

Q: What counts as "one test"?

A: One use of the `(test)` form. Each test should verify one specific behavior.

Q: Should we implement full error handling?

A: Basic error handling (unbound variables, parse errors) is required. Sophisticated error messages are optional but appreciated.